

Kamui Analysis Framework v0.1

An Introductory Guide + Tutorial

Gianfranco de Castro (Cornell)

The Design Philosophy

- Kamui provides a CLI to run everything we want to do
 - Querying DAS, submitting ntuple jobs, etc..
- All the physics is stored within configuration files that live in [kamui/config](#)
 - Sample names, ntuple presets, etc..
- The [protected master branch](#) is only modifiable through PRs that undergo review
- Tags of stable releases will be published
 - [v0.1](#) is covered in these slides
- Bugs or To-Dos are reported in the repo's [issues tab](#)!

Example of running the *help* command

```
bash-5.1$ ./kamui --help
```



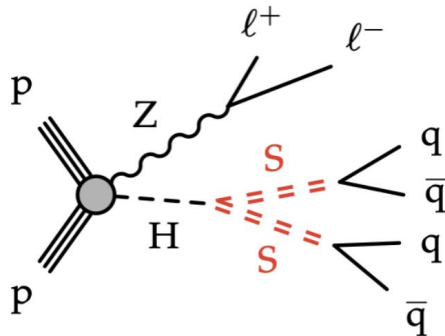
Kamui - the CLI for the analysis framework.
See [python/kamui/README.md](#) for what each command does and the flags they take.

Positional arguments:

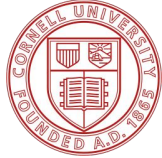
list	Show all available samples
content	Show presets declaring what we save in ntuples
query	Query DAS for how many files, events and GB each selected sample holds
find	Unrestricted DAS dataset search
stage	Copy raw MiniAOD to EOS
submit	Produce ntuples by submitting to condor or CRAB
resubmit	Rerun failed jobs
status	Status of a submitted task
select	Apply an event selection to ntuples
cutflow	Print the cutflow for a given task
norm	Measure and store weights for normalization
check	Validate configuration files
cache	Inspect or clear the DAS cache

The Route We Will Follow

- Our goal is to follow one sample from start to finish!
 - Lepton-triggered exotic Higgs ZH
 - 4d, mS = 55 GeV, $c\tau = 1$ mm, 2018
 - [DAS link](#)
- Validate Kamui against [JMTucker](#)
 - DAS reports 49,999 events
 - $\Rightarrow$ 24,535 after triggers and offline selections
- We will test each part of the Kamui CLI!



1. Initial framework setup - *check*
2. Registering a sample
 - a. *find and norm*
3. Working with a registered sample
 - a. *list, query, stage, cache*
4. Decide what we want to store and register triggers
 - a. *content*
5. Produce the ntuples
 - a. *submit $\Rightarrow$ status $\Rightarrow$ resubmit*
6. Write and apply our offline selection
 - a. *select & cutflow*



Initial Framework Setup

- Git clone [the framework](#) into your LPC workspace (ex: `/uscms/home/USER/nobackup/work`)
- Set-up CMSSW and initialize your grid proxy (see [kamui/README.md](#))
 - **CMSSW_16_1_2** can be installed anywhere (latest CMSSW, not updating unless necessary)
- Every subdirectory has two forms of documentation:
 - **README.md** - What lives here and how to use it (designed for humans to read as reference)
 - **CLAUDE.md** - Why it is built this way and potential pitfalls (designed for AI to read as reference)

[kamui/README.md](#)

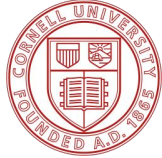
Quick start - CMSSW

First-time setup only! Create a CMSSW_16_1_2 release area in a convenient location:

```
source /cvmfs/cms.cern.ch/cmsset_default.sh
export SCRAM_ARCH=e19_amd64_gcc13
cmsrel CMSSW_16_1_2
```

Then per session, from that release's `src/` :

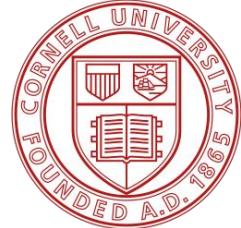
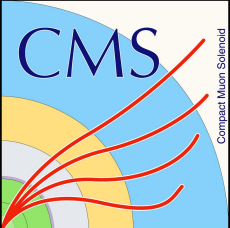
```
source /cvmfs/cms.cern.ch/cmsset_default.sh
export SCRAM_ARCH=e19_amd64_gcc13
cmsenv
voms-proxy-init --rfc --voms cms -valid 192:00
```



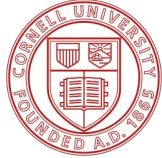
Kamui *check* Command

- Should be run after editing any configuration files or before submitting a batch of jobs (completely local)
- **Config Checks:** Validate the configuration files we write
 - (Catalog, Content Presets, Selections, Triggers, Era isolation, Generator sums, Sites and CMSSW)
- **Architecture Checks:** Reads the source code and checks some rules that are easy to break
 - (Layering and Config Access)
- For the future: Can be incorporated as a CI job after each push, in addition to a manual check!

```
bash-5.1$ ./kamui check
✓ Catalog      Every sample is listed once and is well-formatted (144 samples in 5 families)
✓ Content presets Validated the format of each preset (8/8)
✓ Selections    Validated the format of every cut (8/8)
✓ Triggers      Every trigger file is parseable (2/2)
✓ Era isolation Run 2 and Run 3 keep separate copies of content files
✓ Layering      foundations/ does not import from the rest of the framework
✓ Config access configReaders/ is the only folder containing config access scripts
⚠ Generator sums Samples that know their total generator weight (79/144)
✓ Sites and CMSSW Validated sites.json and the CMSSW job script configurations
```



Registering a Sample

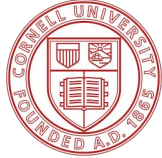


Kamui *find* Command

- The first thing we want to do with our sample, is see if and where it exists!
 - This is a tutorial, so we already know it exists ([DAS link](#))... but what if we didn't know?
- The Kamui *find* command is effectively a wrapper around the [dasgoclient](#); uses the same syntax!
- There are two functionalities added on top of it:
 - **--instance** argument lets you query `prod/global` for official datasets (default), `prod/phys03` for USER ones
 - I implemented a local cache that speeds up queries done before. Use **--refresh** to bypass the cache!

Example of narrowing down a query

```
bash-5.1$ ./kamui find '/*HToSSTodddd*/MINIAODSIM' | wc -l
905
bash-5.1$ ./kamui find '/ZH_HToSSTodddd*/MINIAODSIM' | wc -l
206
bash-5.1$ ./kamui find '/ZH_HToSSTodddd_ZToLL_MH-125_MS-55_ctauS-1_*/MINIAODSIM' | wc -l
28
bash-5.1$ ./kamui find '/ZH_HToSSTodddd_ZToLL_MH-125_MS-55_ctauS-1_*/RunIISummer20UL18*/MINIAODSIM' | wc -l
6
bash-5.1$ ./kamui find '/ZH_HToSSTodddd_ZToLL_MH-125_MS-55_ctauS-1_TuneCP5_13TeV-powheg-pythia8/RunIISummer20UL18MiniAODv2-106X_upgrade2018_realistic_v16_L1v1-v2/MINIAODSIM'
/ZH_HToSSTodddd_ZToLL_MH-125_MS-55_ctauS-1_TuneCP5_13TeV-powheg-pythia8/RunIISummer20UL18MiniAODv2-106X_upgrade2018_realistic_v16_L1v1-v2/MINIAODSIM
1 dataset(s)
```



Registering a Sample to the Framework

So that our framework knows which samples we want to process, we register them in JSON files!

- Anything starting with a `_` is a comment
 - JSON does not natively support comments
- ***family***: A handle to select samples (default is file name)
 - Ex: Exotic Higgs, RPV SUSY, etc..
- Defaults are field values every sample inherits
 - ***era***: Data-taking year
 - ***isMC***: Data vs MC
 - ***dasInstance***: Private vs central sample
 - ***content***: Points to another set of config files detailing what we save to ntuples
 - More later!
 - ***tags***: User-defined tags identifying the sample
- ***samples***: Directly register the sample by DAS path and assign it a name we will use in the framework

Kamui/config/samples/tutorial/zhLeptonTriggered.json

```
{
  "_doc": "Sample for the tutorial!",
  "family": "tutorial",
  "defaults": {
    "era": "2018",
    "isMC": true,
    "dasInstance": "prod/global",
    "content": "dvLepton",
    "tags": ["tutorial", "run2"]
  },
  "samples": [
    {
      "name": "tutorialZH2018",
      "dataset": "/ZH_HToSSTodddd_ZToLL_MH-125_MS-55_ctauS-1"
    }
  ]
}
```

Registering a **Grid** of Samples

Let's say we want to register multiple samples following a pattern

We can use formatted strings! [Kamui/config/samples/exoticHiggs4d2024.json](#)

```
"grids": [
  {
    "_doc": "6 production modes x mS {15,40,55} GeV x ctau {0.1,1,10} mm = 54 points.",
    "name": "{mode}-2S-4D_mS{mS}_ctau{ctau}mm_2024",
    "dataset": "/{dasMode}Hto2Sto4D_Par-ctauS-{ctau}-MH-125-MS-{mS}_TuneCP5_13p6TeV_pow"
```

We can fill the formatted strings using *axes*

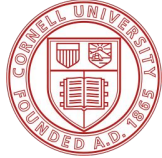
- **mode** defines the Higgs production mode
 - A list of blocks! Fills two placeholders at once.
- **mS** and **ctau** define the parameters of the sample
 - Plain lists! Fills a single entry
 -

For expert usage:

overrides is an argument you can add to the JSONs that adds specific behavior to certain samples

```
"axes": {
  "mode": [
    {
      "mode": "ggH",
      "dasMode": "GluGluH-"
    },
    {
      "mode": "VBFH",
      "dasMode": "VBFH-"
    },
    {
      "mode": "WplusH",
      "dasMode": "WplusH_"
    }
  ],
```

```
"mS": [
  "15",
  "40",
  "55"
],
"ctau": [
  "0p1",
  "1",
  "10"
]
```



Normalization Information + Kamui *norm* Command

[Kamui/config/normalizations/exoticHiggs.json](#)

Registers the cross-sections for processes
Also includes filter efficiencies where needed!

```
"ZH": {  
  "xsec": 0.07788,  
  "_derivation": "3 x 2.596e-02, qq-initiated ZH to llH, total ZH per flavor",  
  "source": "https://gitlab.cern.ch/LHCHIGGSXS/LHCHXSWG1/crosssections"  
},
```

[Kamui/config/normalizations/generatorSums.json](#)

Includes sumGenWeight to normalize samples
Filled w/ *norm*; use when adding new samples!
Documentation for *norm*:

[Kamui/python/kamui/README.md](#)

```
"tutorialZH2018": {  
  "genEvents": 49999,  
  "sumGenWeight": 3966.4178707000005,  
  "sumGenWeight2": 355.13954632751484  
}
```

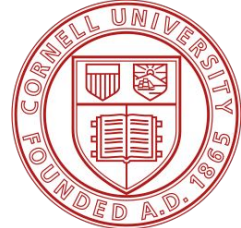
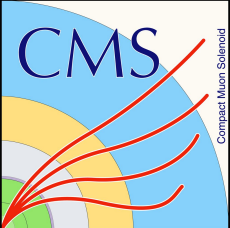
[Kamui/config/normalizations/lumi.json](#)

Luminosities recorded per era

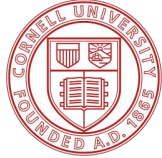
- Run 2 comes from JMTucker
- Run 3 comes from brilcalc commands; exact commands are recorded in the JSON.

Includes functionality to override lumi for specific processes/triggers; ex: Run 2 displacement channel

```
"2018": {  
  "lumi": 59561.0,  
  "source": "JMTucker MFVNeutralino/interface/AnalysisConstants.h, int_lumi_2018"  
},  
"Summer22": {  
  "lumi": 8086.0,  
  "source": "brilcalc lumi -b \"STABLE BEAMS\" -u /pb --normtag /cvmfs/cms-bril.c"
```



Working With Registered Samples



Kamui *list* Command

- We have our sample registered to the framework, what can we do now?
- Kamui builds a catalog of all of our samples that is accessible using different commands and filters.
 - See [Kamui/python/kamui/README.md](#) for the full documentation!

```
bash-5.1$ ./kamui list --era 2018

run2Validation (6 samples)
  Sample                               Era  Content                               Tags
  WminusHToSSTo4d_mS55_ctau1mm_2018  2018  dvLepton                             run2,validation,exoticHiggs,leptonTriggered
  WplusHToSSTo4d_mS55_ctau1mm_2018    2018  dvLepton                             run2,validation,exoticHiggs,leptonTriggered
  ZHToSSTo4d_mS55_ctau1mm_2018        2018  dvLepton                             run2,validation,exoticHiggs,leptonTriggered
  ggHToSSTo4d_mS55_ctau1mm_2018       2018  dvDisplaced                         run2,validation,exoticHiggs,displacementTriggered
  rpvStopDD_M400_ctau1mm_2018         2018  dvDisplaced                         run2,validation,rpv,displacementTriggered
  rpvStopDD_M600_ctau1mm_2018         2018  dvDisplaced                         run2,validation,rpv,displacementTriggered

tutorial (1 sample)
  Sample                               Era  Content                               Tags
  tutorialZH2018                       2018  dvLepton                             tutorial,run2
```

```
bash-5.1$ ./kamui list --family tutorial
```

```
tutorial (1 sample)
  Sample                               Era  Content                               Tags
  tutorialZH2018                       2018  dvLepton                             tutorial,run2
```

```
bash-5.1$ ./kamui list --match '*ZH2018*'
```

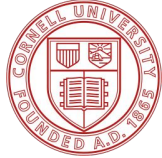
```
tutorial (1 sample)
  Sample                               Era  Content                               Tags
  tutorialZH2018                       2018  dvLepton                             tutorial,run2
```

```
bash-5.1$ ./kamui list --tag tutorial
```

```
tutorial (1 sample)
  Sample                               Era  Content                               Tags
  tutorialZH2018                       2018  dvLepton                             tutorial,run2
```

```
bash-5.1$ ./kamui list --tag rpv --era 2018

run2Validation (2 samples)
  Sample                               Era  Content                               Tags
  rpvStopDD_M400_ctau1mm_2018         2018  dvDisplaced                         run2,validation,rpv,displacementTriggered
  rpvStopDD_M600_ctau1mm_2018         2018  dvDisplaced                         run2,validation,rpv,displacementTriggered
```



Kamui *query* Command

- We have our sample registered to the framework, what can we do now?
- Kamui builds a catalog of all of our samples that is accessible using different commands and filters.
 - See [Kamui/python/kamui/README.md](#) for the full documentation!

```
bash-5.1$ ./kamui query --tag tutorial
```

Sample	Files	Events	Size/GB
tutorialZH2018	20	49,999	3.7
TOTAL	20	49,999	3.7

```
bash-5.1$ ./kamui query --family exoticHiggs4d2024 --match *ggH*
```

Sample	Files	Events	Size/GB
ggH-2S-4D_mS15_ctau0p1mm_2024	146	5,958,672	393.9
ggH-2S-4D_mS15_ctau1mm_2024	144	5,969,256	399.9
ggH-2S-4D_mS15_ctau10mm_2024	133	5,780,076	392.3
ggH-2S-4D_mS40_ctau0p1mm_2024	144	5,868,636	389.5
ggH-2S-4D_mS40_ctau1mm_2024	150	5,963,600	401.2
ggH-2S-4D_mS40_ctau10mm_2024	155	5,945,700	403.5
ggH-2S-4D_mS55_ctau0p1mm_2024	154	5,893,280	392.5
ggH-2S-4D_mS55_ctau1mm_2024	151	5,946,646	400.0
ggH-2S-4D_mS55_ctau10mm_2024	151	5,981,575	405.0
TOTAL	1328	53,307,441	3577.9

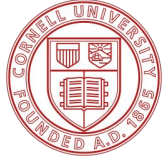
```
bash-5.1$ ./kamui query --era 2018
```

Sample	Files	Events	Size/GB
rpvStopDD_M400_ctau1mm_2018	5	50,000	3.3
rpvStopDD_M600_ctau1mm_2018	10	36,000	2.6
ggHToSSTo4d_mS55_ctau1mm_2018	39	546,940	35.6
ZHToSSTo4d_mS55_ctau1mm_2018	20	49,999	3.7
WplusHToSSTo4d_mS55_ctau1mm_2018	19	49,994	3.3
WminusHToSSTo4d_mS55_ctau1mm_2018	2	49,995	3.2
tutorialZH2018	20	49,999	3.7
TOTAL	115	832,927	55.5



```
bash-5.1$ ./kamui query --tag rpv --era 2018
```

Sample	Files	Events	Size/GB
rpvStopDD_M400_ctau1mm_2018	5	50,000	3.3
rpvStopDD_M600_ctau1mm_2018	10	36,000	2.6
TOTAL	15	86,000	5.9



Kamui cache and stage Commands

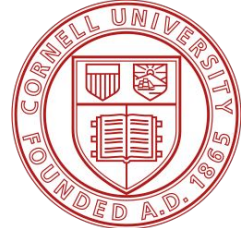
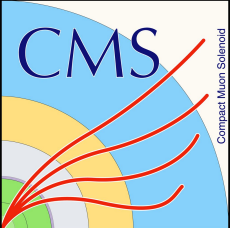
- We have our sample registered to the framework, what can we do now?
- Kamui builds a catalog of all of our samples that is accessible using different commands and filters.
 - See [Kamui/python/kamui/README.md](#) for the full documentation!

```
bash-5.1$ ./kamui cache
230 cached DAS responses, 0.8 MB in /uscms_data/d3/
age      0.0 to 3.0 days
expired  0 (older than 30 days, ignored on read)
```

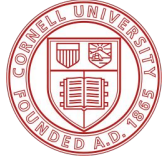
- The *cache* stores previous DAS queries to avoid repetitive queries that eat runtime.
- Add *--prune* to get rid of “expired” entries
- Add *--clear* to get rid of all entries

- *Stage* files from DAS into EOS for local tests
- Add *--dryRun* to see what would happen but not copy
- Add *--maxFiles* to copy a subset of the entire dataset

```
bash-5.1$ ./kamui stage --name ggH-2S-4D_mS55_ctau10mm_2024 --dryRun --maxFiles 1
=== ggH-2S-4D_mS55_ctau10mm_2024
    /GluGluH-Hto2Sto4D_Par-ctauS-10-MH-125-MS-55_TuneCP5_13p6TeV_powheg-pythia8/RunIII
un3_2024_realistic_v2-v2/MINIAODSIM
    151 files in DAS, copying 1
    [dryRun] xrd fs root://cmseos.fnal.gov mkdir -p /store/user/lpcdisplacedvertices/gg
ctau10mm_2024
    [dryRun] xrd cp root://cms-xrd-global.cern.ch//store/mc/RunIII2024Summer24MiniAODV
0-MH-125-MS-55_TuneCP5_13p6TeV_powheg-pythia8/MINIAODSIM/150X_mcRun3_2024_realistic
9-a41d-981c781103e0.root root://cmseos.fnal.gov//store/user/lpcdisplacedvertices/gg
ctau10mm_2024/126bcda7-1a5e-4069-a41d-981c781103e0.root
```



Defining Our Ntuples



Defining Collections

[Kamui/config/content/README.md](#)
[Kamui/config/content/run2/collections/...](#)

- How can we turn DAS samples into Ntuples that have the information we want?
- We don't want to save the entire DAS file locally - they are huge!
 - So let's define the important collections we want to save locally

JSON header + object collection definition

```
"_doc": "Generator-level content. All mcOnly.",
"collections": {
  "GenPart": {
    "type": "genParticle",
    "src": "prunedGenParticles",
    "mcOnly": true,
    "doc": "Pruned generator particles",
    "variables": {
      "pt": {
        "expr": "pt()",
        "type": "float",
        "doc": "pT [GeV]",
        "precision": 10
      }
    }
  },
```

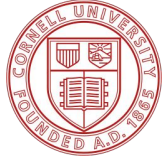
Global definition

```
"Rho": {
  "type": "global",
  "doc": "Pileup energy densities",
  "variables": {
    "fastjetAll": {
      "src": "fixedGridRhoFastjetAll",
      "type": "double",
      "doc": "Rho from all PF candidates"
    },
    "fastjetCentral": {
      "src": "fixedGridRhoFastjetCentral",
      "type": "double",
      "doc": "Rho, |eta| < 2.5"
    }
  }
}
```

Save trigger bits!

```
"triggerBits": {
  "processes": [
    "HLT",
    "PAT",
    "RECO"
  ]
}
```

Note: also have the ability to apply skims; e.g. seed tracks

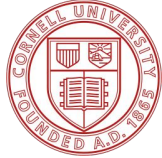


Defining Triggers

- How can we turn DAS samples into Ntuples that have the information we want?
- Let's define the trigger paths we use
 - For now, we just include the Run II Low HT analysis' lepton- and displacement-triggered paths

```
{
  "_doc": "Run 2 lepton-triggered channel.",
  "process": "HLT",
  "mode": "any",
  "paths": [
    "HLT_Ele27_WPTight_Gsf_v*",
    "HLT_Ele35_WPTight_Gsf_v*",
    "HLT_Ele32_WPTight_Gsf_v*",
    "HLT_Ele115_CaloIdVT_GsfTrkIdT_v*",
    "HLT_Ele50_CaloIdVT_GsfTrkIdT_PFJet165_v*",
    "HLT_IsoMu27_v*",
    "HLT_IsoMu24_v*",
    "HLT_Mu50_v*"
  ],
}
```

```
{
  "_doc": "Run 2 displacement-triggered channel.",
  "process": "HLT",
  "mode": "any",
  "paths": [
    "HLT_QuadJet45_TripleBTagCSV_p087_v*",
    "HLT_DoubleJet90_Double30_TripleBTagCSV_p087_v*",
    "HLT_DoubleJetsC100_DoubleBTagCSV_p014_DoublePFJetsC100MaxDeta1p6_v*",
    "HLT_DoublePFJets100MaxDeta1p6_DoubleCaloBTagCSV_p33_v*",
    "HLT_PFHT300PT30_QuadPFJet_75_60_45_40_TriplePFBTagCSV_3p0_v*",
    "HLT_DoublePFJets116MaxDeta1p6_DoubleCaloBTagDeepCSV_p71_v*",
    "HLT_PFHT330PT30_QuadPFJet_75_60_45_40_TriplePFBTagDeepCSV_4p5_v*",
    "HLT_HT350_DisplacedDijet40_DisplacedTrack_v*",
    "HLT_HT650_DisplacedDijet80_Inclusive_v*",
    "HLT_HT430_DisplacedDijet40_DisplacedTrack_v*",
    "HLT_HT650_DisplacedDijet60_Inclusive_v*"
  ],
}
```



Defining Content

- How can we turn DAS samples into Ntuples that have the information we want?
- Let's define the content we want to save in different situations

Collections

```
{ } core.json
{ } gen.json
{ } jets.json
{ } leptons.json
{ } tracks.json
{ } vertices.json
```

dvBase.json

```
"_doc": "Event,
"include": [
  "core",
  "jets",
  "leptons",
  "vertices"
]
```

+gen

dvSignal.json

```
"_doc": "Adds
"include": [
  "dvBase",
  "gen"
]
```

+tracks

dvFull.json

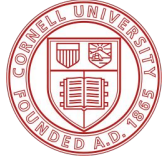
```
"_doc": "Adds
"include": [
  "dvSignal",
  "tracks"
]
```

dvDisplaced.json

```
"_doc": "Displacement-triggered
"include": [
  "dvFull"
],
"skim": {
  "triggers": "run2Displaced"
}
```

dvLepton.json

```
"_doc": "Lepton-triggered
"include": [
  "dvFull"
],
"skim": {
  "triggers": "run2Lepton"
}
```

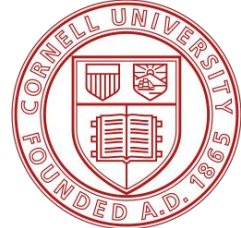
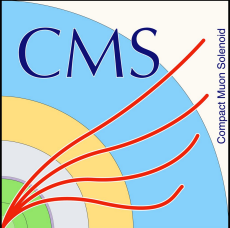


Kamui *content* Command

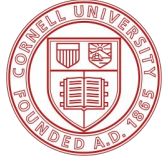
- Kamui allows you to visualize what each content configuration file will save
- You can also include the **--write** flag to save the example configuration file fed into the jobs locally

```
bash-5.1$ ./kamui content dvLepton --era 2018
dvLepton (isMC=True)
  Collection  Type           Source                                Vars
  BeamSpot    beamSpot        offlineBeamSpot                      8
  CaloJet      candidate       slimmedCaloJets                     4  maxLen=40
  Electron     patElectron     slimmedElectrons                   22  cut='pt > 5' maxLen=20
  GenMET       patMET          slimmedMETs                        2
  GenPart      genParticle     prunedGenParticles                11
  Jet          patJet          slimmedJets                        19  cut='pt > 20 && abs(eta)
  LostTrack    packedCandidate lostTracks                         16  cut='charge != 0 && hasT
  MET          patMET          slimmedMETs                        5
  Muon         patMuon         slimmedMuons                       23  cut='pt > 5' maxLen=20
  PV           vertex          offlineSlimmedPrimaryVertices      11
  Pileup       pileup          slimmedAddPileupInfo              0
  PuppiMET     patMET          slimmedMETsPuppi                  3
  Rho          global          2
  SV           secondaryVertex slimmedSecondaryVertices           11  maxLen=30
  Track        packedCandidate packedPFCandidates                 19  cut='charge != 0 && hasT
  genWeight    genWeight       generator                          0

16 collections, 156 variables
skim: run2Lepton (8 paths, mode any, process HLT)
```



Job Submission



Testing a Job by Hand

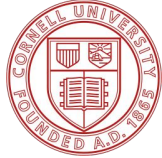
- We are ready to submit a job! What should we do to prepare for job submission, and how do we do it?
- Before submitting a million jobs, it is a good idea to test your inputs and ensure they work locally!

1. Write the content preset out to a local path
 - a. Ex: `./kamui content dvLepton --era 2018 --write /tmp/dvLepton.json`
2. Run cmsRun using this preset and an input file
 - a. Example command shown on the right for the tutorial file with a max of 1000 events!
 - b. Look at the header prints as they are very informative.

```
cmsRun ntupleProduction/cmssw/kamuiNtuple_cfg.py \  
  content=/tmp/dvLepton.json \  
  isMC=True \  
  inputFiles=root://cms-xrd-global.cern.ch//store  
  outputFile=test.root \  
  maxEvents=1000
```

```
[kamui] content preset : dvLepton (isMC=True)  
[kamui] output          : test_numEvent1000.root  
[kamui] collections     : BeamSpot, CaloJet, Electron, GenMET, GenPart, Jet, LostTrack, MET,  
                        Muon, PV, Pileup, PuppMET, Rho, SV, Track, genWeight  
[kamui] HLT skim        : HLT_Ele27_WPTight_Gsf_v*, HLT_Ele35_WPTight_Gsf_v*, ... 8 paths
```

3. Inspect that the output file is of the format you expect :)



Submitting a Job

- We are ready to submit a job! What should we do to prepare for job submission, and how do we do it?
- Now, how do we actually submit a job?

The [Kamui *submit* command](#) takes care of all of your needs!

It is fully documented in [Kamui/python/kamui/README.md](#)

First, do a dry run to see what would happen

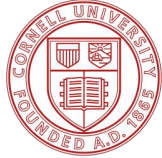
```
bash-5.1$ ./kamui submit --family tutorial --task tutorial --dryRun

Task 'tutorial' : 1 sample(s), backend=condor
tutorialZH2018 20 file(s)
wrote 4 job(s) under Kamui/ntupleProduction/jobs/tutorial
output goes to /store/user/lpcdisplacedvertices/gdecastr/ntuples/tutorial
[dryRun] condor_submit ../jobs/tutorial/submit.jdl
```

Then, perform the actual job submission

```
bash-5.1$ ./kamui submit --family tutorial --task tutorial

Task 'tutorial' : 1 sample(s), backend=condor
tutorialZH2018 20 file(s)
wrote 4 job(s) under Kamui/ntupleProduction/jobs/tutorial
output goes to /store/user/lpcdisplacedvertices/gdecastr/ntuples/tutorial
Submitting job(s)....
4 job(s) submitted to cluster 85329939.
Attempting to submit jobs to lpcschedd6.fnal.gov
wrote provenance to root://cmseos.fnal.gov//store/.../ntuples/tutorial/task.json
```



Monitoring and Resubmission

- We are ready to submit a job! What should we do to prepare for job submission, and how do we do it?
- Once we've submitted a job, how do we check that it is running as intended, and resubmit it if it failed?

The Kamui [status](#) and [resubmit](#) command takes care of all of your needs!

They are fully documented in [Kamui/python/kamui/README.md](#)

Check the status of a submitted job

```
bash-5.1$ ./kamui status --task tutorial
```

ID	OWNER	SUBMITTED	RUN_TIME	ST	PRI	SIZE	CMD
85329939.0	gdecastr	9/2 12:32	0+00:25:04	R	0	765.0	runJob_dvLepton_mc_2018.sh tutorialZH2018 0

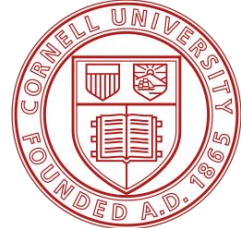
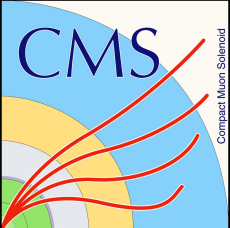
Total for query: 1 jobs; 0 completed, 0 idle, 1 running, 0 held

Task tutorial (condor)
Samples 1, content dvLepton
Jobs 4
From 753359b7 on master (dirty tree), by gdecastr at 2026-09-02T12:32:35-05:00
Output /store/user/lpcdisplacedvertices/gdecastr/ntuples/tutorial
Record Kamui/ntupleProduction/jobs/tutorial/task.json

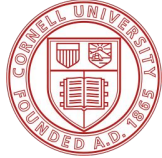
Resubmit a failed job (in this case none needed)

```
bash-5.1$ ./kamui resubmit --task tutorial --dryRun
```

Task tutorial (condor)
Output /store/user/lpcdisplacedvertices/gdecastr/ntuples/tutorial
Outputs 3/4 present on EOS
Missing 1 job(s):
tutorialZH2018 job 0
1 job(s) from this task are still queued or running.
refusing to resubmit while they run. Wait, or pass --forceResubmit to submit anyway.



Applying Selections



Building a Selection

- Our jobs are ready! How do we apply offline selections?
- First, we need to build our selection JSONs, which define the selections we want to apply.

Define the JSON with our selections. For the lepton JSON, we have the MET filters and lepton offline requirements.

```
"cuts": [
  {
    "name": "metFilters",
    "type": "flags",
    "flags": {
      "2016APV": [
        "Flag_goodVertices",
        "Flag_globalSuperTightHalo2016Filter",
        "Flag_HBHENoiseFilter",
        "Flag_HBHENoiseIsoFilter",
        "Flag_EcalDeadCellTriggerPrimitiveFilter",
        "Flag_BadPFMuonFilter",
        "Flag_BadPFMuonDzFilter",
        "Flag_hfNoisyHitsFilter",
        "Flag_eeBadScFilter"
      ]
    }
  }
],
```

```
"doc": "Muon leg",
"requirements": [
  {
    "variable": "pt",
    "min": {
      "2016APV": 27,
      "2016": 27,
      "2017": 30,
      "2018": 27
    }
  },
  {
    "variable": "eta",
    "absMax": 2.4
  },
  {
    "variable": "mediumId",
    "min": 1
  },
  {
    "variable": "pfRelIso04",
    "max": 0.15
  },
  {
    "variable": "dxyBeamspot",
    "absMax": 0.02
  },
  {
    "variable": "dzPV",
    "absMax": 0.5
  }
]
```

They can also be trigger-gated!

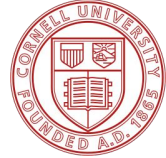
[Example in the disp. channel](#)

```
"name": "quadJet45TripleBTag",
"eras": [
  "2016APV",
  "2016"
],
"doc": "2016 tri-b-tag quad-jet path: four jets above
"cuts": [
  {
    "name": "fired",
    "type": "trigger",
    "triggers": [
      "HLT_QuadJet45_TripleBTagCSV_p087_v*"
    ],
    "doc": "HLT_QuadJet45_TripleBTagCSV_p087 fired"
  },
  {
    "name": "fourJets",
    "type": "quantity",
    "doc": "At least four identified jets",
    "conditions": [
      {
        "quantity": "nJet20",
        "min": 4
      }
    ]
  }
],
```

```
{
  "name": "threeBTags",
  "type": "object",
  "doc": "Three loose b-tags",
  "anyOf": [
    {
      "collection": "Jet",
      "min": 3,
      "doc": "Three loose-tagged jets",
      "requirements": [
        {
          "variable": "pt",
          "min": 20
        },
        {
          "variable": "eta",
          "absMax": 2.5
        },
        {
          "variable": "tightLepVeto",
          "min": 1
        },
        {
          "variable": "btagDeepFlavB",
          "min": {
            "2016APV": 0.0508,
            "2016": 0.048
          }
        }
      ]
    }
  ]
}
```

Applying Selections and Building Cutflows

[Kamui/python/kamui/README.md](https://github.com/Kamui/python/kamui/README.md)



- Our jobs are ready! How do we apply offline selections?
- Now, let's apply the selections to our ntuples and construct some informative cutflows.

The [Kamui *select* command](#) applies the cuts and makes a new ntuple

```
bash-5.1$ ./kamui select --family tutorial --selection run2Lepton --task tutorialSel --inputTask tutorial

Task 'tutorialSel' : 1 sample(s), selection=run2Lepton
tutorialZH2018                28,299 ->  24,535  (86.7%)
cutflow written under Kamui/ntupleSelection/out/tutorialSel
```

The [Kamui *cutflow* command](#) prints the corresponding cutflow

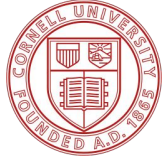
```
bash-5.1$ ./kamui cutflow --task tutorialSel

task tutorialSel  selection run2Lepton

tutorialZH2018

```

cut	type	events	removed	step eff	cumulative
generated		49,999	0	100.00%	100.00%
input		28,299	21,700	56.60%	56.60%
metFilters	flags	28,270	29	99.90%	56.54%
leptonPlateau	object	24,535	3,735	86.79%	49.07%



Conclusions

- We are done! 🎉
- If you have been following the tutorial sample all the way through, you should have **24,535 events**.
 - This is an exact match with JMTucker!
- I have myself gone through and validated all 4 years in both the lepton- and displacement-triggered channels
 - Every case is an exact match!
- Please provide feedback on this framework through these channels:
 - Mattermost messages to Gianfranco de Castro
 - Emails to gd396@cornell.edu
 - Opening issues on the [repository](#)
- Thank you for following these slides :)